

String class in Java

25 August 2026 20:35

In Java, String is basically an object that represents sequence of char values. An array of characters works as a string in Java, for example:

```
char c[] = {'J', 'a', 'v', 'a'};
c
<object reference> (char[])
String s = new String(c);
s
"Java" (String)
```

Same as:

```
String s1 = "Java";
s1
"Java" (String)
```

Java String class provides a lot of methods to perform operations on String such as compareTo(), concat(), equals(), split(), length(), substring(), etc.

Generally, a **String** is a sequence of characters. However, a String in Java is an object that represents a sequence of characters. And the java.lang.String class is used to create String objects. Java strings are *immutable*. That means their value cannot be changed once created.

There are two ways to create string object:

- String literal
- Using new keyword

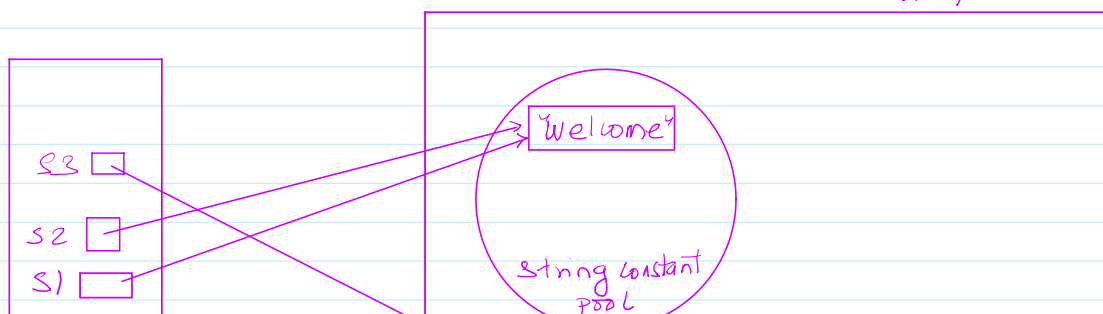
Creating string using string literal

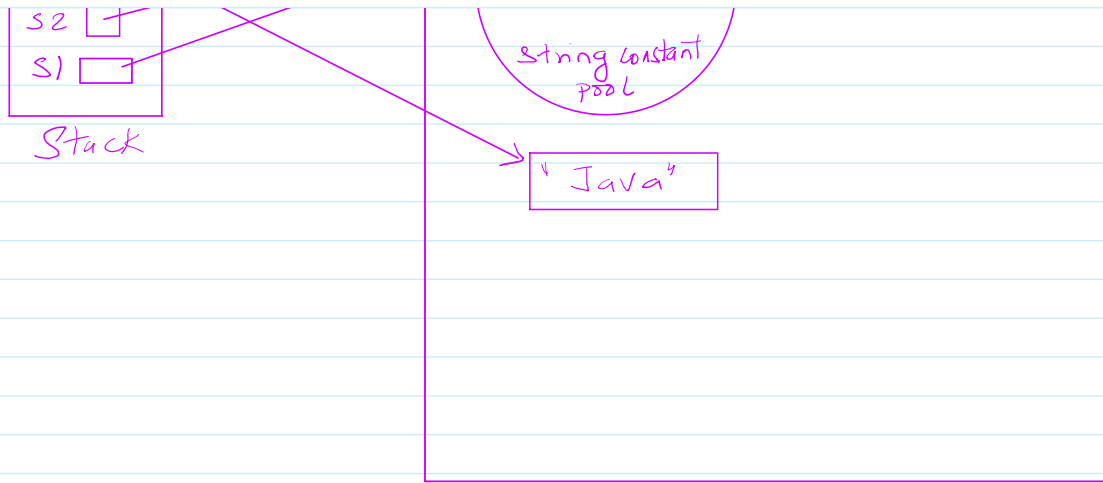
Java String literal is created by using double quotes. Example:

```
String s1 = "Java";
```

Each time you create a string literal, the JVM checks the string constant pool first. If the string already exist in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new String instance is created and placed in the pool. For example:

```
→String s1 = "Welcome";
→String s2 = "Welcome"; // it doesn't create a new instance
```





In the above example, only one object will be created. Firstly, JVM will not find any string object with the value welcome in the String constant pool. That is why it will create a new object. After that it will find the string with the value welcome in the pool. It will not create a new object but will return the reference to the same instance.

2. Creating string using new keyword

A string can be created using the new keyword, which explicitly creates a new String object in memory rather than using the string literal pool.

For example:

```
String s3 = new String("Java");
```

Here,

- "Java" Is a string literal, so it is stored in the string pool (one object).
- "new String("Java") Create a new String object in the heap (second object)
- s3 Is the reference variable that points to the object in the heap.

In such case JVM will create a new String object in normal (non-pool) heap memory and the literal "Welcome" will be placed in the String constant pool. The variable s3 will refer to the object in a heap (non-pool).

```
String s = new String("Java");
```

The screenshot shows the IntelliJ IDEA IDE with two panels. The left panel displays the String object's internal state, including the 'value' field (a byte array), 'coder', 'hash', and 'hashIsZero'. The right panel shows the 'value' field as a byte array, with the length set to 4. The byte array contains the ASCII values of the characters 'J', 'a', 'v', and 'a'. A handwritten note '4 bytes' points to the byte array. Below the panels, the variable 's' is shown with the value 'Java' (String).

```
char[] c = {'J', 'a', 'v', 'a'};
```

`char[] c = {'J', 'a', 'v', 'a'};`
↑
2 bytes for each cell. $2 \times 4 = 8$ bytes occupied

Finding String Length

You can find the number of characters in a string by calling the `length()` method on the string object.

```
s
"Java" (String)
int len = s.length();
len
4 (int)
```

Finding a Character in a String

You can find a character or substring in a string by using the `indexOf` method and return its position.

```
int index = s.indexOf("a");
```

$s \rightarrow$

0	1	2	3
J	a	v	a

```
index
1 (int)
```

Returning each character from the string.

```
char c = s.charAt(index);
```

```
package MyStringCodes;
import java.util.*;
```

```
/**
 * Write a description of class StringDemo1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringDemo1
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter any sentence: ");
        String str = sc.nextLine();
        int i;
        for(i = 0; i < str.length(); i++){
            char c = str.charAt(i);
            System.out.print("\n" + c);
        }
    }
}
```

Output:

```
BlueJ: Terminal Window - Learning_Java
Options
Enter any sentence: Hello! Java

H
e
l
l
o
!

J
a
v
a
```

```
package MyStringCodes;
import java.util.*;
```

```
/**
 * Write a description of class NumberOfConsonantsAndVowels here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class NumberOfConsonantsAndVowels
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter any sentence: ");
        String str = sc.nextLine();
        int i, v = 0, conso = 0;
        String vowels = "AEIOUaeiou";
        String la="a", ua = "A";
        for(i = 1; i < 26; i++){
            //concatenation means joing of two strings.
            la += (char)(la.charAt(i-1) + 1); //explicit type casting
            ua += (char)(ua.charAt(i-1) + 1); //explicit type casting
        }
        System.out.print("\n" + la + "\n" + ua + "\n");
        for(i = 0; i < str.length(); i++){
            char c = str.charAt(i);
            if((la.indexOf(c) != -1 || ua.indexOf(c) != -1)){
                if(vowels.indexOf(c) > -1){
                    v++;
                }
                else{
                    conso++;
                }
            }
        }
        System.out.print("\nVowels count: " + v);
        System.out.print("\nC consonant count: " + conso);
    }
}
```

This function search the the value passed in the parameter, in the given string, if found, returns the index of first occurrence, otherwise returns '-1'.

Output:

```
BlueJ: Terminal Window - Learning_Java
Options
Enter any sentence: Welcome! Java

abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ

Vowels count: 5
Cosonant count: 6
```

length()
charAt()
indexOf()

String comparison

Strings can be compared based on their **content** or their **references**. String comparison checks whether two strings have the same sequence of characters, while a reference comparison checks whether both variables point to the same object in memory.

The following are the different ways to compare strings:

- Using equals() method
- Using compareTo() method
- Using == Operator
- Using startsWith() and endsWith() methods

Using equals() method: The equals () method compares the original content of the string. It compares the values of the string for equality. If both the strings are equal then it returns true otherwise false.

- public boolean equals(Object another): compares this string to the specified object.
- public boolean equalsIgnoreCase(Object another): compares this string to the specified object, and ignore the cases of alphabet.

boolean endsWith (String)	
boolean equals (Object)	
boolean equalsIgnoreCase (String)	

boolean equals (Object anObject)
Compares this string to the specified object. The result is <code>true</code> if and only if the argument is not <code>null</code> and is a <code>String</code> object that represents the same sequence of characters as this object.
For finer-grained String comparison, refer to {@link java.text.Collator} .
Parameters
anObject - The object to compare this <code>String</code> against
return - <code>true</code> if the given object represents a <code>String</code> equivalent to this string, <code>false</code> otherwise
see - #compareTo(String)
see - #equalsIgnoreCase(String)

```
package MyStringCodes;
import java.util.*;
```

```
/**
 * Write a description of class StringComparison1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringComparison1
```

```

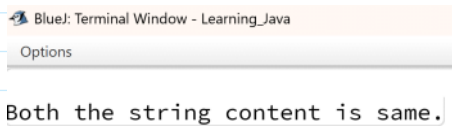
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String str1, str2;
        str1 = new String("Java");
        str2 = new String ("JAVA");

        if(str1.equals("Java")){
            System.out.print("\nBoth the string content is same.");
        }
        else{
            System.out.print("\nContent is not same!");
        }
    }
}

```

String parameter in Java it is represented by this keyword

Output:



```

BlueJ: Terminal Window - Learning_Java
Options

Both the string content is same.

```

boolean equalsIgnoreCase(String) Related: boolean contentEquals(CharSequence) boolean contentEquals(StringBuffer) boolean equals(Object)	boolean equalsIgnoreCase(String anotherString) Compares this String to another String, ignoring case considerations. Two strings are considered equal ignoring case if they are of the same length and corresponding Unicode code points in the two strings are equal ignoring case. Two Unicode code points are considered the same ignoring case if at least one of the following is true: - The two Unicode code points are the same (as compared by the == operator) - Calling Character.toLowerCase(Character.toUpperCase(int)) on each Unicode code point produces the same result Note that this method does <i>not</i> take locale into account, and will result in unsatisfactory results for certain locales. The (@link java.text.Collator)
--	--

```

package MyStringCodes;
import java.util.*;

/**
 * Write a description of class StringComparison1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringComparison1
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        String str1, str2;
        str1 = new String("Java");
        str2 = new String ("JAVA");
        if(str1.equals("Java")){
            System.out.print("\nBoth the string content is same.");
        }
    }
}

```

```

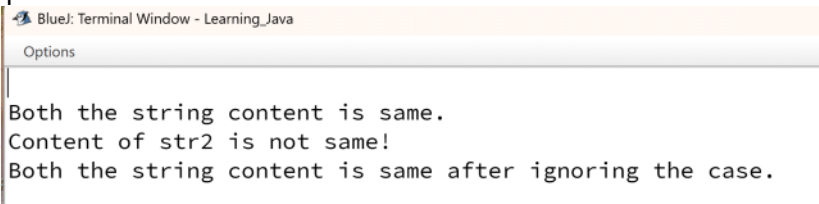
        else{
            System.out.print("\nContent is not same!");
        }

        if(str1.equals(str2)){
            System.out.print("\nBoth the string content is same.");
        }
        else{
            System.out.print("\nContent of str2 is not same!");
        }

        if(str1.equalsIgnoreCase(str2)){
            System.out.print("\nBoth the string content is same after ignoring the
case.");
        }
        else{
            System.out.print("\nContent of str2 is not same!");
        }
    }
}

```

Output:



```

Blue: Terminal Window - Learning_Java
Options
Both the string content is same.
Content of str2 is not same!
Both the string content is same after ignoring the case.

```

```

package MyStringCodes;
import java.util.*;

/**
 * Write a description of class StringComparison1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringComparison1
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        String str1, str2, str3, str4, str5;
        str1 = new String("Java");
        str2 = new String ("JAVA");
        str3 = new String("Java");
        if(str1.equals("Java")){
            System.out.print("\nBoth the string content is same.");
        }
        else{
            System.out.print("\nContent is not same!");
        }
    }
}

```

```

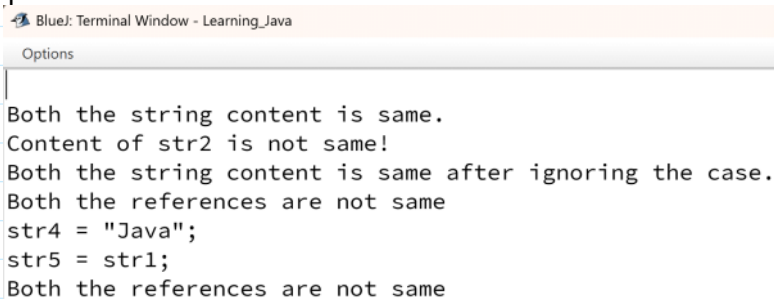
        if(str1.equals(str2)){
            System.out.print("\nBoth the string content is same.");
        }
        else{
            System.out.print("\nContent of str2 is not same!");
        }

        if(str1.equalsIgnoreCase(str2)){
            System.out.print("\nBoth the string content is same after ignoring the
case.");
        }
        else{
            System.out.print("\nContent of str2 is not same!");
        }

        if(str1 == str3){
            System.out.print("\nBoth the references are same");
        }
        else{
            System.out.print("\nBoth the references are not same");
        }
        System.out.print("\nstr4 = \"Java\";\nstr5 = str1;");
        str4 = "Java";
        str5 = str1;
        // str5 = "Java";
        if(str4 == str5){
            System.out.print("\nBoth references are same");
        }
        else{
            System.out.print("\nBoth the references are not same");
        }
    }
}

```

Output:



```

Blue: Terminal Window - Learning_Java
Options
Both the string content is same.
Content of str2 is not same!
Both the string content is same after ignoring the case.
Both the references are not same
str4 = "Java";
str5 = str1;
Both the references are not same

```

```

package MyStringCodes;
import java.util.*;

```

```

/**
 * Write a description of class StringComparison1 here.
 *
 * @author (your name)

```

```

* @version (a version number or a date)
*/
public class StringComparison1
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        String str1, str2, str3, str4, str5;
        str1 = new String("Java");
        str2 = new String ("JAVA");
        str3 = new String("Java");
        if(str1.equals("Java")){
            System.out.print("\nBoth the string content is same.");
        }
        else{
            System.out.print("\nContent is not same!");
        }

        if(str1.equals(str2)){
            System.out.print("\nBoth the string content is same.");
        }
        else{
            System.out.print("\nContent of str2 is not same!");
        }

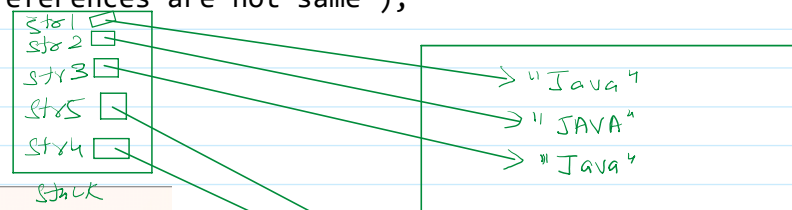
        if(str1.equalsIgnoreCase(str2)){
            System.out.print("\nBoth the string content is same after ignoring the
case.");
        }
        else{
            System.out.print("\nContent of str2 is not same!");
        }

        if(str1 == str3){
            System.out.print("\nBoth the references are same");
        }
        else{
            System.out.print("\nBoth the references are not same");
        }
        // System.out.print("\nstr4 = \"Java\";\nstr5 = str1;");
        str4 = "Java";
        // str5 = str1;
        str5 = "Java";
        System.out.print("\nstr4 = \"Java\";\nstr5 = \"Java\";");
        if(str4 == str5){
            System.out.print("\nValue stored in string constant pool");
            System.out.print("\nBoth references are same");
        }
        else{
            System.out.print("\nBoth the references are not same");
        }
    }
}

```

Output:

BlueJ: Terminal Window - Learning_Java

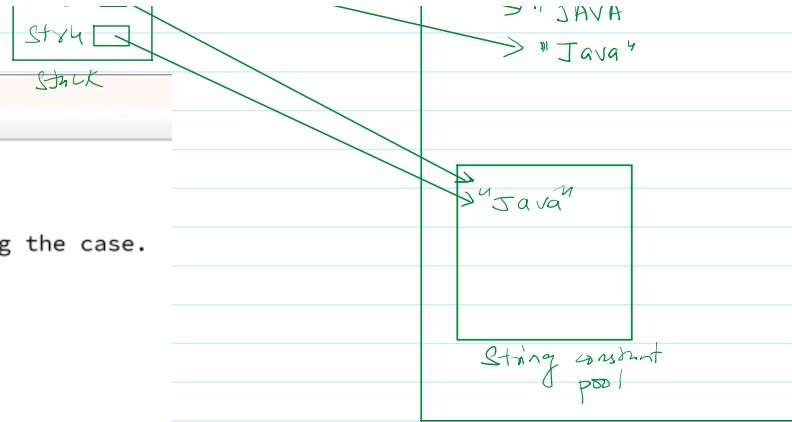


Output:

BlueJ: Terminal Window - Learning_Java

Options

```
Both the string content is same.
Content of str2 is not same!
Both the string content is same after ignoring the case.
Both the references are not same
str4 = "Java";
str5 = "Java";
Value stored in string constant pool
Both references are same
```



compare
↑ ↑ ↑ ↑

Using compareTo() method

The string compareTo() method compares values lexicographically. And returns an integer value that describe if first string is less than, equal to or greater than the 2nd string.

Suppose S1 and S2 are two String objects if:

- S1 == S2: The method returns zero.
- S1 > S2: The method returns a positive value.
- S1 < S2: The method returns a negative value.

s1 → Raman
s2 → Ramesh
s1.compareTo(s2); 97 - 101 = -4

s2.compareTo(s1) → 101 - 97 = 101

```
package MyStringCodes;
```

```
/**
 * Write a description of class CompareToMethod here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CompareToMethod
{
    public static void main(String[] args){
        String str1 = new String("Tom");
        String str2 = new String("Tom");
        String str3 = new String("Harry");
        System.out.print("\f"+ str1.compareTo(str2)); // 0
        System.out.print("\n"+ str1.compareTo(str3)); // str1 > str3
        System.out.print("\n"+ str3.compareTo(str2)); // str3 < str2
    }
}
```

Output:

BlueJ: Terminal Window - Learning_Java

Options

```
0
12
-12
```

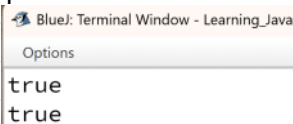
Using startsWith and endsWith() methods

If a string starts with a prefix and ends with a suffix, respectively, it may be determined using the methods startsWith() and endsWith();

```
package MyStringCodes;
```

```
/**
 * Write a description of class StartsWithAndEndsWith here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StartsWithAndEndsWith
{
    public static void main(String[] args){
        String str = "String Compare";
        System.out.print("\f" + str.startsWith("String")); // true
        System.out.print("\n" + str.endsWith("Compare")); //true
    }
}
```

Output:



```
Blue: Terminal Window - Learning_Java
Options
true
true
```

String Concatenation in Java

String concatenation is a basic and commonly used operation that allows us to combine 2 or more strings into a single string. Since strings are **immutable**, any operation that tries to "change" a string actually ends up creating a new string in memory. It is widely used for creating messages, formatting output and working with text data.

Some of the ways to perform string concatenation are:

1. Using '+' operator (String Concatenation Operator)
2. Using concat() method of String class.
3. Using String.format() method

1. Using string concatenation operator

The simplest and most commonly used method for concatenating string is using the + operator. We can concatenate two or more string by using the + operator between them.

Example:

```
String n1 = "Mathew";
String n2 = "Hadyen";
String fn = n1 + " " + n2;
fn
"Mathew Hadyen" (String)
n1
"Mathew" (String)
n2
```

string + int → string
string + int → string

```

n1
"Mathew"    (String)
n2
"Hadyen"    (String)

String s = 40 + 30 + fn + 40 + 40;

String s = 40 + 30 + fn + 40 + 40;
s
"70Mathew Hadyen4040"    (String)

```

2. Using the `String.concat()` method

The `String.concat()` method is an alternative of the string concatenation operator (+) for joining two or more strings. It appends one string to the end of another.

Syntax:

```
public String concat(String another)
```

Example:

```

String x = n1.concat(n2);
x
"MathewHadyen"    (String)
x = n1.concat(" ").concat(n2);
x
"Mathew Hadyen"    (String)
n1
"Mathew"    (String)
n2
"Hadyen"    (String)

```

3. Using the `String.format()` method: The `String.format()` method provides another way to concatenate strings in Java while allowing for more control over the formatting.

Official Documentation:

```
public static String format(String format, Object... args)
```

Returns a formatted string using the specified format string and arguments.

The locale always used is the one returned by `Locale.getDefault(Locale.Category.FORMAT)` with `FORMAT` category specified.

Parameters:

format - A [format string](#)

args - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by The Java Virtual Machine Specification. The behavior on a null argument depends on the [conversion](#).

Returns:

A formatted string

Throws:

[IllegalFormatException](#) - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the [Details](#) section of the formatter class

specification.

Since:

1.5

Conversion	Argument Category	Description
'b', 'B'	general	If the argument <i>arg</i> is null, then the result is "false". If <i>arg</i> is a boolean or <code>Boolean</code> , then the result is the string returned by <code>String.valueOf(arg)</code> . Otherwise, the result is "true".
'h', 'H'	general	The result is obtained by invoking <code>Integer.toHexString(arg.hashCode())</code> .
's', 'S'	general	If <i>arg</i> implements <code>Formattable</code> , then <code>arg.formatTo</code> is invoked. Otherwise, the result is obtained by invoking <code>arg.toString()</code> .
'c', 'C'	character	The result is a Unicode character
'd'	integral	The result is formatted as a decimal integer
'o'	integral	The result is formatted as an octal integer
'x', 'X'	integral	The result is formatted as a hexadecimal integer
'e', 'E'	floating point	The result is formatted as a decimal number in computerized scientific notation
'f'	floating point	The result is formatted as a decimal number
'g', 'G'	floating point	The result is formatted using computerized scientific notation or decimal format, depending on the precision and the value after rounding.
'a', 'A'	floating point	The result is formatted as a hexadecimal floating-point number with a significand and an exponent. This conversion is not supported for the <code>BigDecimal</code> type despite the latter's being in the <i>floating point</i> argument category.
't', 'T'	date/time	Prefix for date and time conversion characters. See Date/Time Conversions .
'%'	percent	The result is a literal '%' ('\u0025')
'n'	line separator	The result is the platform-specific line separator

Any characters not explicitly defined as conversions are illegal and are reserved for future extensions.

The format specifiers for general, character, and numeric types have the following syntax:

`%[argument_index$][flags][width][.precision]conversion`

The optional *argument_index* is a decimal integer indicating the position of the argument in the argument list. The first argument is referenced by "1\$", the second by "2\$", etc.

The optional *flags* is a set of characters that modify the output format. The set of valid flags depends on the conversion.

The optional *width* is a positive decimal integer indicating the minimum number of characters to be written to the output.

The optional *precision* is a non-negative decimal integer usually used to restrict the number of characters. The specific behavior depends on the conversion.

The required *conversion* is a character indicating how the argument should be formatted. The set of valid conversions for a given argument depends on the argument's data type.

From <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/Formatter.html#syntax>

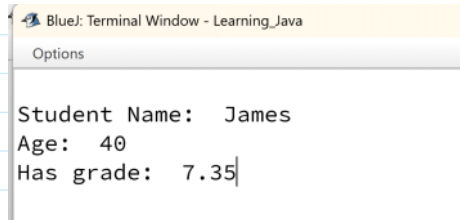
Example:

```
package MyStringCodes;
```

```
/**
 * Write a description of class StringFormatDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StringFormatDemo
{
```

```
public static void main(String args[]){
    System.out.print("\f");
    double d = 7.3458;
    int age = 40;
    String nm = "James";
    String nn = String.format("%s %s %n%s %d %n%s %.2f", "Student Name: ", nm, "Age:
",age, "Has grade: ", d );
    System.out.print("\n" + nn);
}
}
```

Output:

A screenshot of a terminal window titled "BlueJ: Terminal Window - Learning_Java". The terminal shows the output of the Java program: "Student Name: James", "Age: 40", and "Has grade: 7.35". The text is displayed on three separate lines, with a blank line between the first and second lines, and another blank line between the second and third lines, matching the format specifiers in the code. The cursor is positioned at the end of the third line.

```
BlueJ: Terminal Window - Learning_Java
Options

Student Name:  James
Age:  40
Has grade:  7.35|
```